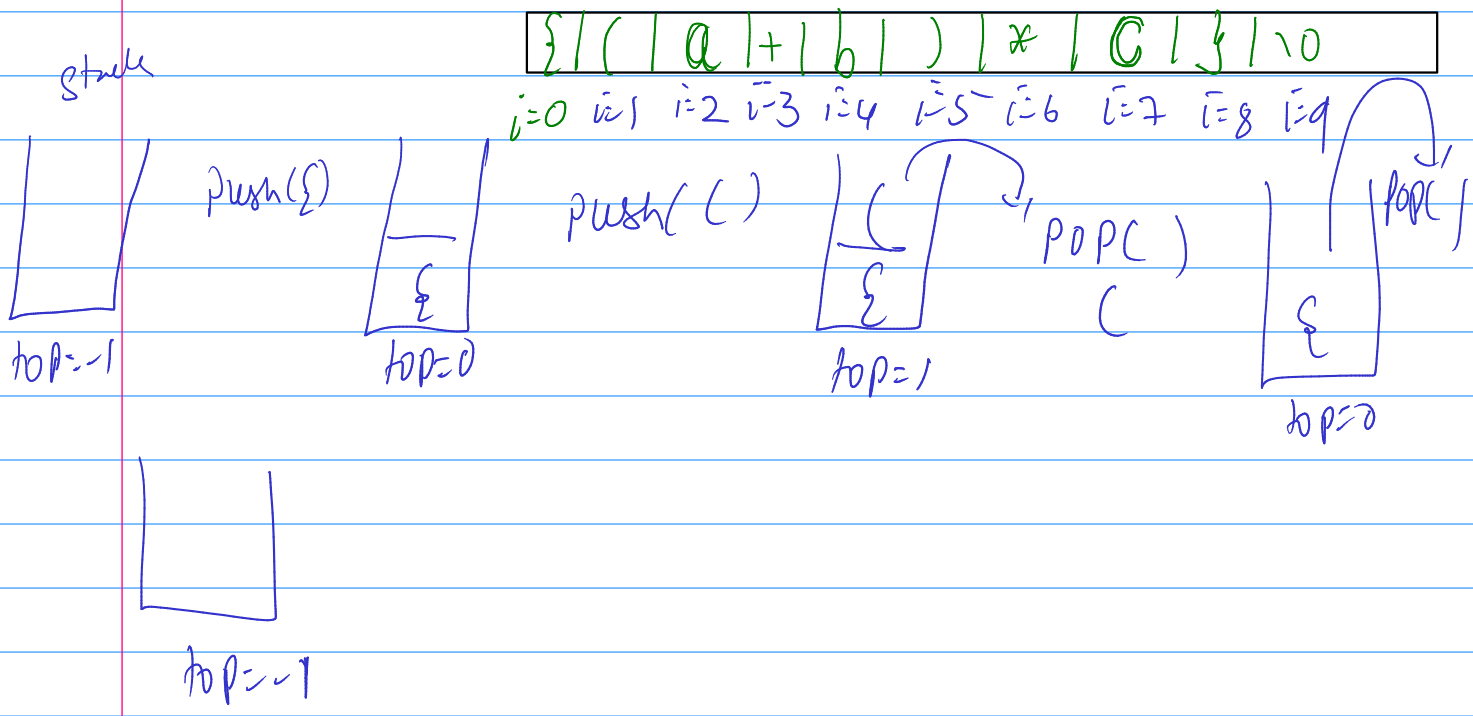
5/04

Checking for balanced parenthesis

1. traverse from left to right of the expression
2. if current element is openbrace push on to stack
3. else if current is curly closed brace pop the stack if return d is not the curly open brace itsnot abalanced so terminate the lo
4. else if curretn is function closed brace pop the stack, if return data is not the function open brace termiante the loop
5. else if current element is square closed brace pop the stack if return data is not equal to square open brace termiante the l
6. i++ continue the loop.
7. after termianting the loop if we reached to \0 in the string and stack is empty then its a balanced
else unbalanced

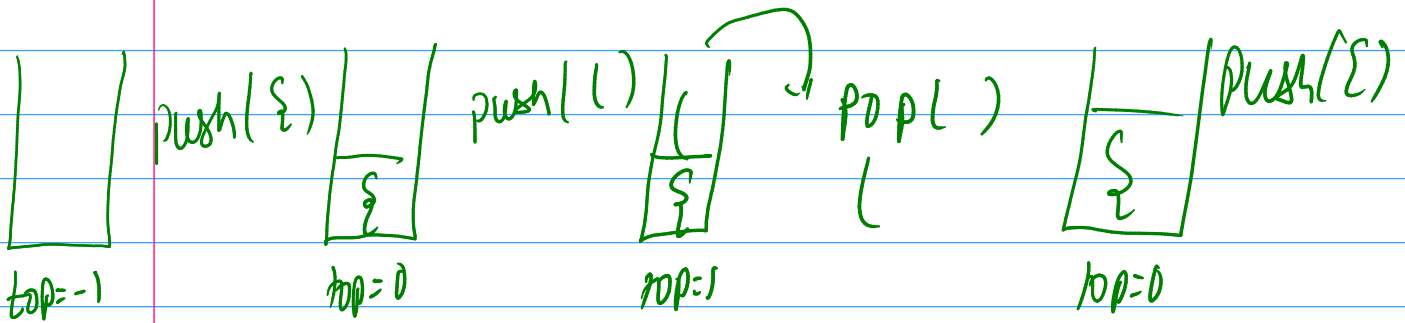
if(pop() != '(')
brcals

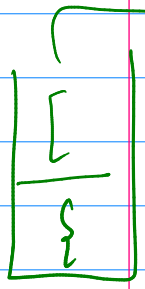
char s[]="{(a+b)*c}"



char s[7]="{(a+b)*[c-d}]";

{	(	a	+	b	)	*	[	c	-	d	}	]	\0
i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10	i=11		





pop() [!= { ✓

we execute break;

((a+b) (d+b))];

infix , prefix and postfix expressions

in infix expression we place operator between the operands.
ex :

A+B
A+B*C
(A+B)*C

prefix
in prefix expression we place operator before the operands
ex :

+AB
+A*BC
*+ABC

postfix
in postfix expression we place the operator after the operands
ex : AB+
ABC*+
AB+C*

infix to postfix conversion

1. traverse the expression from left to right.
2. if current elememt is open brace push on to stack.
3. else if current element is operator, push on to stack if any of the above conditions becomes true
    a. stack is empty
    b. top of the stack pointing to an open brace
    c. top of the stack pointing to an operator whose priority is less than current operator
4. else pop the stack unitil we get any of the abvove conditions tru

Note : poped data should be concatenate to postfix expression

5. if current element is closed , pop the stack unitil we get its equivalent open brace.

Note : poped data should concate to postfix expression

6. else concatenate to postfix
7. Once loop terminated, pop the stack unitl it becomes empty

Note : Poped data should be concatenate to postfix